



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	ONG-IKO, KATELYN
Section:	AN43
Professor:	MS. CRISOLA TAN
Date:	10/9/26

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF file**.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                             random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
 - **Number of agents** (e.g., 5)
 - **Grid size** (e.g., 10 for a 10x10 grid)
 - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

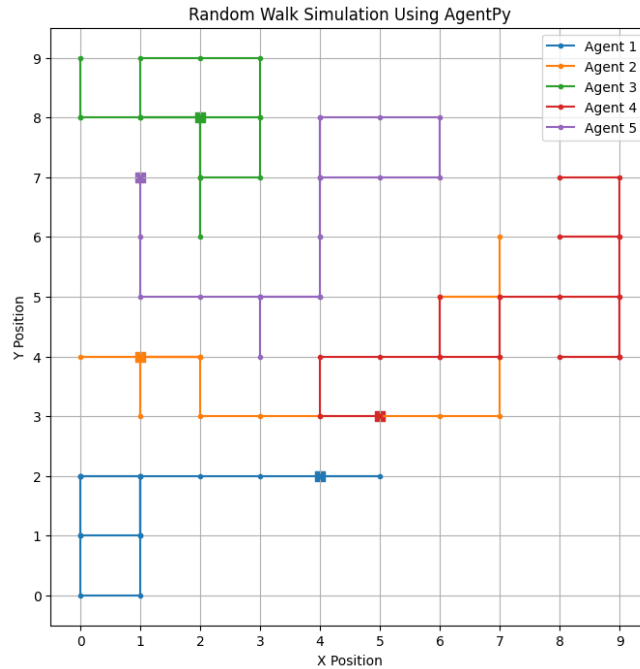
1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
 - How agents spread across the grid over time
 - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

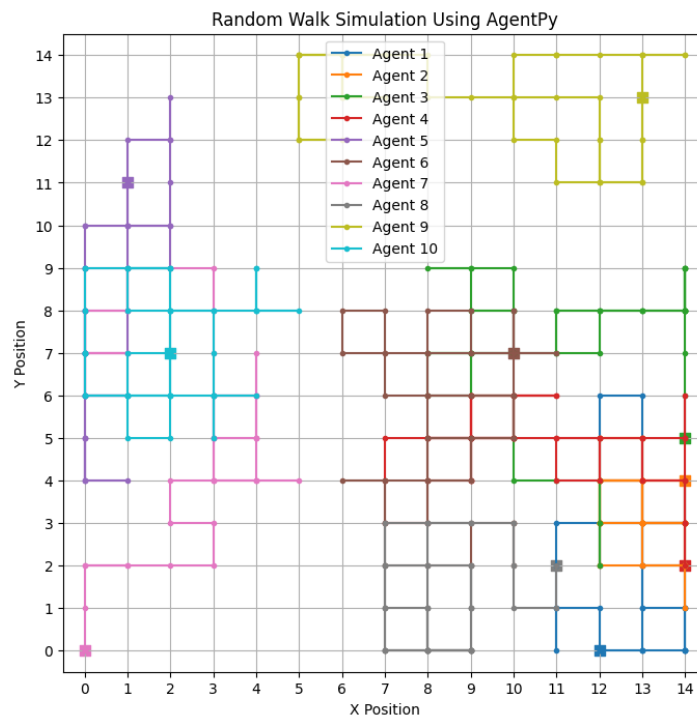
- Repeat the simulation with **different values**:
 - **More agents** (e.g., 10 or 20)
 - **Smaller/larger grid sizes**

- Longer simulations (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Experiment 1: AGENTS: 5, GRID SIZE: 10, SIMULATION: 20 STEPS



Experiment 2: AGENTS: 10, GRID SIZE: 15, SIMULATION: 50 STEPS



Laboratory Tasks

1. **Track agent paths:** Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules:** Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions:** Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios:** Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ↴

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
from collections import Counter

# -----
# AGENT DEFINITION
# -----
class RandomWalker(ap.Agent):

    def setup(self):
        size = self.model.p.grid_size

        self.position = (
            random.randint(0, size - 1),
            random.randint(0, size - 1)
        )

        # Task 1: Record the agent's path
        self.path = [self.position]

    def step(self):
        size = self.model.p.grid_size
        x, y = self.position

        # Possible movement directions
        directions = [
            (1, 0), (-1, 0), (0, 1), (0, -1)
        ]

        # Task 2: Preferred direction
        if self.model.p.movement_rule == "preferred":
```

```

        directions = [
            (1, 0), (1, 0), (1, 0),
            (-1, 0), (0, 1), (0, -1)
        ]

    random.shuffle(directions)

    # Try possible movements
    for dx, dy in directions:
        nx = max(0, min(size - 1, x + dx))
        ny = max(0, min(size - 1, y + dy))

        # Task 2: Avoid occupied positions
        if self.model.p.avoid_others:
            occupied = [
                agent.position
                for agent in self.model.walkers
                if agent is not self
            ]

            if (nx, ny) in occupied:
                continue

        self.position = (nx, ny)
        break

    # Record the new position
    self.path.append(self.position)

# -----
# MODEL DEFINITION
# -----
class RandomWalkModel(ap.Model):

    def setup(self):
        self.walkers = ap.AgentList(
            self, self.p.agents, RandomWalker
        )

    def step(self):
        for agent in self.walkers:
            agent.step()

```

```

# -----
# SIMULATION FUNCTION
# -----
def run_simulation(
    agents=5,
    grid_size=10,
    steps=20,
    movement_rule="random",
    avoid_others=False
):
    parameters = {
        "agents": agents,
        "grid_size": grid_size,
        "steps": steps,
        "movement_rule": movement_rule,
        "avoid_others": avoid_others
    }

    model = RandomWalkModel(parameters)
    model.setup()

    for _ in range(steps):
        model.step()

    return model

# -----
# TASK 1: TRACK AGENT PATHS
# -----
model = run_simulation(
    agents=5,
    grid_size=10,
    steps=20
)

plt.figure(figsize=(8, 8))

for i, agent in enumerate(model.walkers):
    x = [p[0] for p in agent.path]
    y = [p[1] for p in agent.path]

    plt.plot(x, y, marker="o", markersize=3,

```

```

        label=f"Agent {i + 1}")
    plt.scatter(x[0], y[0], marker="s", s=60)

plt.title("Task 1: Agent Paths")
plt.xlabel("X Position")
plt.ylabel("Y Position")
plt.xticks(range(10))
plt.yticks(range(10))
plt.xlim(-0.5, 9.5)
plt.ylim(-0.5, 9.5)
plt.grid(True)
plt.legend()
plt.show()

# -----
# TASK 2: CHANGE MOVEMENT RULES
# -----
avoid_model = run_simulation(
    agents=5,
    grid_size=10,
    steps=20,
    movement_rule="random",
    avoid_others=True
)

preferred_model = run_simulation(
    agents=5,
    grid_size=10,
    steps=20,
    movement_rule="preferred",
    avoid_others=False
)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for ax, simulation, title in [
    (axes[0], avoid_model, "Agents Avoid Each Other"),
    (axes[1], preferred_model, "Preferred Rightward Movement")
]:
    for i, agent in enumerate(simulation.walkers):
        x = [p[0] for p in agent.path]
        y = [p[1] for p in agent.path]
        ax.plot(x, y, marker="o", markersize=2,

```

```

        label=f"Agent {i + 1}")

    ax.set_title(title)
    ax.set_xlabel("X Position")
    ax.set_ylabel("Y Position")
    ax.set_xlim(-0.5, 9.5)
    ax.set_ylim(-0.5, 9.5)
    ax.set_xticks(range(10))
    ax.set_yticks(range(10))
    ax.grid(True)

plt.tight_layout()
plt.show()

# -----
# TASK 3: ANALYZE FINAL POSITIONS
# -----
print("TASK 3: FINAL AGENT POSITIONS")

final_positions = []

for i, agent in enumerate(model.walkers):
    print(f"Agent {i + 1}: {agent.position}")
    final_positions.append(agent.position)

# Count how many agents occupy each final position
position_counts = Counter(final_positions)

print("\nFinal Position Distribution:")
for position, count in sorted(position_counts.items()):
    print(f"Position {position}: {count} agent(s)")

# Plot the final positions
plt.figure(figsize=(7, 7))

x = [agent.position[0] for agent in model.walkers]
y = [agent.position[1] for agent in model.walkers]

plt.scatter(x, y, s=150)

for i, agent in enumerate(model.walkers):
    plt.annotate(
        f"A{i + 1}",

```

```

        agent.position,
        xytext=(5, 5),
        textcoords="offset points"
    )

plt.title("Task 3: Final Agent Positions")
plt.xlabel("X Position")
plt.ylabel("Y Position")
plt.xticks(range(10))
plt.yticks(range(10))
plt.xlim(-0.5, 9.5)
plt.ylim(-0.5, 9.5)
plt.grid(True)
plt.show()

# -----
# TASK 4: COMPARE TWO SCENARIOS
# -----
scenario_a = run_simulation(
    agents=5,
    grid_size=10,
    steps=20
)

scenario_b = run_simulation(
    agents=10,
    grid_size=15,
    steps=50
)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

for ax, simulation, title in [
    (axes[0], scenario_a, "Scenario A: 5 Agents, 10x10 Grid, 20 Steps"),
    (axes[1], scenario_b, "Scenario B: 10 Agents, 15x15 Grid, 50 Steps")
]:
    for agent in simulation.walkers:
        x = [p[0] for p in agent.path]
        y = [p[1] for p in agent.path]
        ax.plot(x, y, alpha=0.8)

    size = simulation.p.grid_size
    ax.set_title(title)

```

```

ax.set_xlabel("X Position")
ax.set_ylabel("Y Position")
ax.set_xlim(-0.5, size - 0.5)
ax.set_ylim(-0.5, size - 0.5)
ax.set_xticks(range(size))
ax.set_yticks(range(size))
ax.grid(True)

```

```

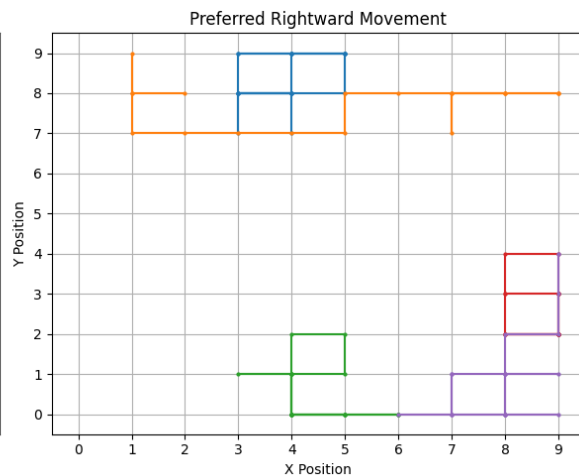
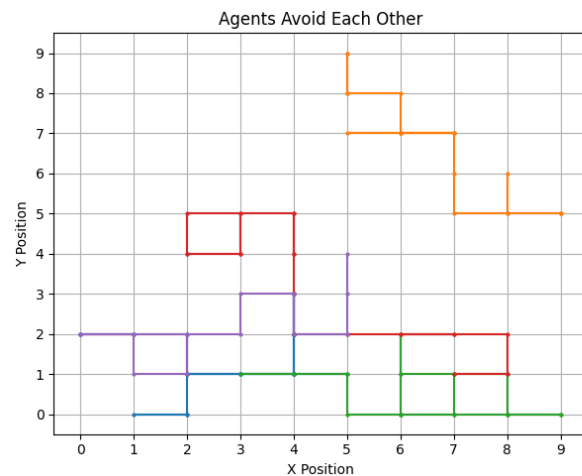
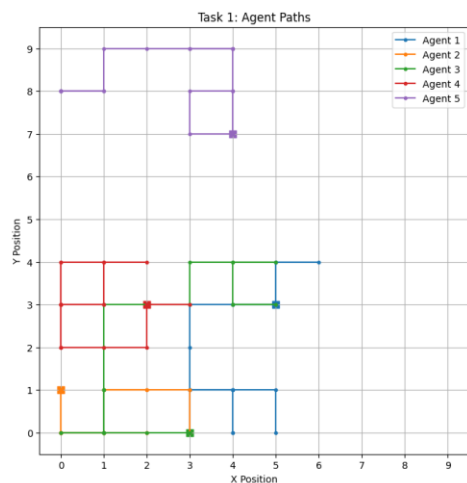
plt.tight_layout()
plt.show()

```

```

print("TASK 4: SCENARIO COMPARISON")
print("Scenario A: 5 agents, 10x10 grid, 20 steps")
print("Scenario B: 10 agents, 15x15 grid, 50 steps")
print("Compare the number of paths, movement distance, and")
print("how widely the agents spread across each grid.")

```



TASK 3: FINAL AGENT POSITIONS

Agent 1: (5, 0)

Agent 2: (3, 0)

Agent 3: (4, 3)

Agent 4: (2, 4)

Agent 5: (0, 8)

Final Position Distribution:

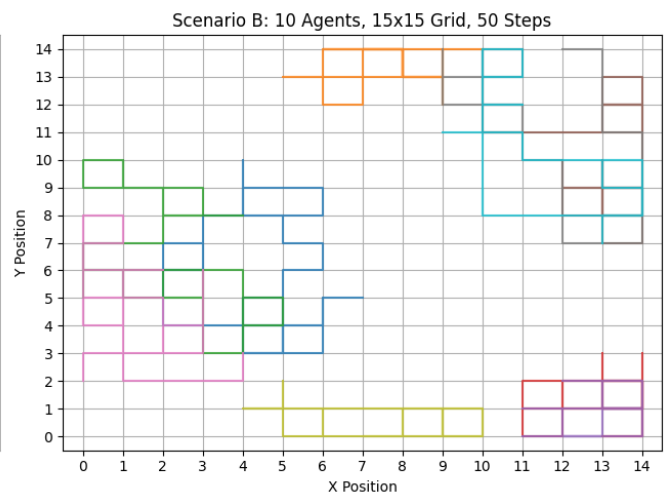
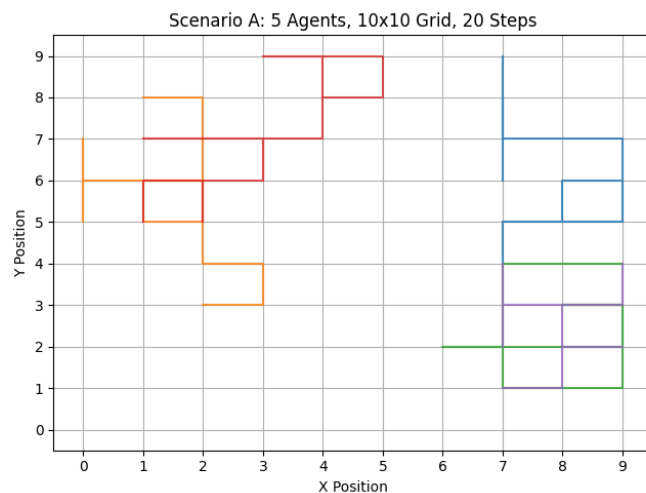
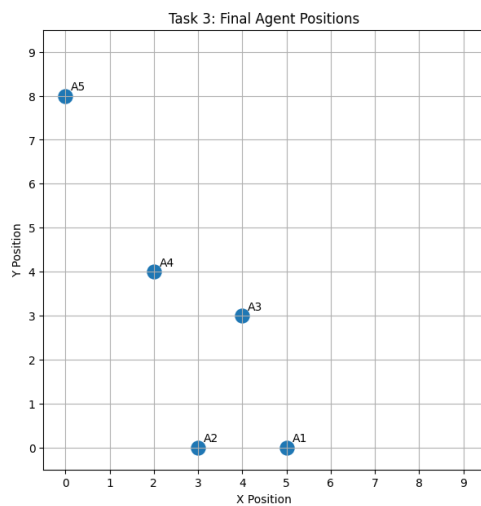
Position (0, 8): 1 agent(s)

Position (2, 4): 1 agent(s)

Position (3, 0): 1 agent(s)

Position (4, 3): 1 agent(s)

Position (5, 0): 1 agent(s)



Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ↓

1. What happens if the number of agents is very high compared to the grid size?

The grid becomes crowded, and more agents may occupy the same positions. If agents are programmed to avoid each other, their movement becomes more restricted.

2. How does increasing the number of steps affect the spread of agents?

Increasing the number of steps allows agents to travel farther and explore more areas of the grid. However, their final distribution remains random.

3. Do the agents ever move outside the grid? Why or why not?

No. The code restricts their X and Y coordinates to the grid boundaries, preventing them from moving outside the grid.

4. How can this simulation be used to model real-life phenomena, such as diffusion or crowd movement?

The simulation can represent how people move through an area or how particles spread randomly over time. Different movement rules can model crowd avoidance or preferred directions.